

CMPE 492
Multimodal Deep Learning for Genomic &
Transcriptomic Data:
Alignment-Free Cancer Detection
Semester 2 — Midterm/Progress Report

Nuri Başar — 2021400129
Osman Selim Yüksel — 2021400105

Advisor:
Assoc. Prof. Mehmet Turan

Spring 2026

Contents

1	INTRODUCTION	1
1.1	Broad Impact	1
1.2	Ethical Considerations	1
2	PROJECT DEFINITION AND PLANNING	2
2.1	Project Definition	2
2.2	Project Planning	2
2.2.1	Project Time and Resource Estimation	2
2.2.2	Success Criteria	3
2.2.3	Risk Analysis	3
2.2.4	Team Work (if applicable)	4
3	RELATED WORK	5
4	METHODOLOGY	6
4.1	Architectural Refactoring & Pipeline Modernization	6
4.2	Patient-Level Aggregation: The Crowd-Voting Methodology	6
4.2.1	Formal Definition	7
4.2.2	Theoretical Justification	7
4.2.3	Implementation	7
4.3	DNABERT-2 Integration & Engineering Challenges	7
4.3.1	Data Engineering: Reverse-Encoding Pipeline	8
4.3.2	Model Architecture	8
4.3.3	Engineering Hurdles Overcome	9
4.3.4	Training Optimization	10
5	REQUIREMENTS SPECIFICATION	11
5.1	Functional Requirements	11
5.1.1	FR1: Data Ingestion and Preprocessing (Updated)	11
5.1.2	FR2: Model Architectures (Updated)	11
5.1.3	FR3: Training and Optimization (Updated)	11
5.1.4	FR4: Evaluation and Explainability (Updated)	11
5.2	Non-Functional Requirements	12
5.3	System Constraints	12
5.4	Validation and Traceability	12
6	DESIGN	13
6.1	Information Structure	13
6.2	Information Flow	13

6.3	System Design	14
6.4	User Interface Design (if applicable)	15
7	IMPLEMENTATION AND TESTING	16
7.1	Implementation	16
7.1.1	Development Environment	16
7.1.2	Key Implementation Details	16
7.2	Testing	17
7.2.1	Unit Testing	17
7.2.2	Integration Testing	17
7.2.3	Performance Testing	17
7.3	Deployment	17
8	RESULTS	18
8.1	Experimental Setup	18
8.2	DNABERT-2 Training Dynamics	18
8.3	Read-Level Performance Comparison	19
8.4	Patient-Level Performance: Crowd-Voting Aggregation	20
8.5	Computational Performance Comparison	22
9	CONCLUSION	23
9.1	Discussion	23
9.1.1	The 80bp Context Limitation	23
9.1.2	Overfitting After Epoch 3	23
9.1.3	Crowd-Voting: The Ultimate Game-Changer	23
9.1.4	Limitations	24
9.2	Summary of Semester 2 Contributions	24
9.3	Future Work	25
A	REPOSITORY STRUCTURE	26

Chapter 1

INTRODUCTION

1.1 Broad Impact

Cancer remains one of the leading causes of mortality worldwide, and early detection from genomic data holds transformative potential for clinical oncology. This project develops an alignment-free deep learning framework that operates directly on raw FASTQ sequencing reads, bypassing the computationally expensive and reference-dependent bioinformatics alignment stage. By demonstrating that weak per-read signals can be aggregated into robust patient-level diagnostics, this work opens a pathway toward rapid, portable cancer screening tools that require neither a reference genome nor specialized bioinformatics expertise.

The broader significance of the Semester 2 contributions extends along three axes. First, the successful integration of a genomic foundation model (DNABERT-2) demonstrates that pre-trained transformer representations are transferable to clinical cancer detection tasks even under severe context constraints. Second, the patient-level aggregation paradigm—which we term *Crowd-Voting*—establishes a principled statistical framework for converting millions of individually noisy read-level predictions into a single high-confidence diagnostic decision. Third, the architectural refactoring of our codebase from monolithic notebooks into a modular, reproducible Python package sets a standard for open-science practices in bioinformatics deep learning research.

1.2 Ethical Considerations

All ethical principles established in Semester 1 continue to govern this work. We use exclusively publicly available, pre-anonymized datasets from the NCBI Sequence Read Archive (SRA) and Gene Expression Omnibus (GEO). No personal health information or clinical identifiers are involved at any stage. Our system is designed as a research tool and does not constitute a clinical diagnostic device; all findings require independent validation through established clinical protocols before any potential medical application.

In Semester 2, we additionally emphasize transparency and reproducibility by open-sourcing the complete codebase, including training scripts, evaluation notebooks, and data engineering utilities, at <https://github.com/osmannselim/Genomic-RawSeq-Analyzer>. We acknowledge the risk that high patient-level AUC values may be misinterpreted as clinical-grade performance; accordingly, we explicitly document dataset size limitations and the need for external validation cohorts.

Chapter 2

PROJECT DEFINITION AND PLANNING

2.1 Project Definition

The overarching objective of this project is to develop a deep learning framework capable of detecting cancer directly from raw FASTQ sequencing data without reliance on reference genome alignment. Semester 1 established the data acquisition pipeline, implemented baseline 1D-CNN and LSTM Autoencoder models, and demonstrated the feasibility of extracting a weak predictive signal from individual 80-base-pair (bp) reads.

Semester 2 builds upon these foundations with three principal objectives:

- (i) **Codebase Modernization:** Refactor the monolithic Colab notebook into a professional, modular Python package to ensure reproducibility and extensibility.
- (ii) **Patient-Level Aggregation:** Implement and validate the Crowd-Voting hypothesis—that aggregating thousands of weak read-level predictions per patient yields a strong diagnostic signal.
- (iii) **Foundation Model Integration:** Replace the lightweight 1D-CNN with DNABERT-2, a 117-million-parameter transformer pre-trained on multi-species genomic data, and rigorously compare both architectures at read-level and patient-level granularity.

2.2 Project Planning

2.2.1 Project Time and Resource Estimation

Phase	Duration	Key Tasks	Resources
Codebase Refactoring	3 weeks	Modularize notebooks into <code>src/</code> package; implement CLI entry points; add configuration management	GitHub, VS Code, Python

Phase	Duration	Key Tasks	Resources
Data Engineering for DNABERT-2	2 weeks	Reverse-encode integer arrays to raw DNA strings; implement BPE-compatible data loaders; validate tokenization	NumPy, HuggingFace Tokenizers
DNABERT-2 Fine-Tuning	3 weeks	Resolve framework incompatibilities; implement mixed-precision training; hyperparameter search	NVIDIA A100, PyTorch, HuggingFace Transformers 4.44.0
Patient-Level Aggregation	2 weeks	Implement group-by aggregation; optimize threshold selection; validate on held-out patients	Pandas, Scikit-learn
Evaluation & Comparison	2 weeks	Side-by-side CNN vs. DNABERT-2 benchmarking at read-level and patient-level	Matplotlib, Scikit-learn
Documentation	2 weeks	Write progress report; update repository README; prepare presentation	L ^A T _E X, GitHub

2.2.2 Success Criteria

The success of Semester 2 is measured against the following targets:

- (i) **Reproducibility:** The entire pipeline—from raw data to evaluation metrics—must be executable from the public GitHub repository with a single configuration file.
- (ii) **Patient-Level AUC > 0.90:** The Crowd-Voting aggregation must elevate the weak read-level signal to a clinically meaningful patient-level AUC exceeding 0.90 for at least one model.
- (iii) **Foundation Model Comparison:** DNABERT-2 must be successfully fine-tuned and quantitatively compared against the 1D-CNN baseline on identical test data.

2.2.3 Risk Analysis

Risk	Impact	Mitigation Strategy
Framework incompatibility with DNABERT-2	High	Pin <code>transformers</code> to version 4.44.0; monkey-patch custom attention layers; disable Triton flash attention
GPU out-of-memory during transformer training	High	Implement mixed-precision (AMP) training; reduce batch size or use gradient accumulation; leverage A100 40 GB

Overfitting of DNABERT-2 on limited data	Medium	Apply early stopping; use dropout in classification head; monitor validation loss divergence
Weak read-level signal insufficient for aggregation	Medium	Increase reads per patient ($\sim 50,000$); explore weighted voting schemes

2.2.4 Team Work (if applicable)

The team division of labor for Semester 2 was as follows:

Nuri Başar: DNABERT-2 model architecture design and training pipeline; resolution of framework compatibility issues (Triton, meta-device, flash attention); mixed-precision optimization; training loop implementation with checkpoint management.

Osman Selim Yüksel: Codebase refactoring and modularization; data engineering for BPE tokenization (reverse-encoding pipeline); patient-level aggregation implementation; evaluation notebook development; visualization and statistical analysis.

Collaborative: Joint debugging sessions for DNABERT-2 integration; co-design of evaluation methodology; shared documentation and report writing.

Chapter 3

RELATED WORK

The related work survey from Semester 1 remains applicable and is summarized here with Semester 2-specific additions.

Classical machine learning methods (Random Forests, XGBoost) have demonstrated strong performance on structured genomic features but require extensive feature engineering and cannot process raw sequencing reads. Deep learning approaches, particularly 1D-CNNs and LSTMs, introduced the ability to learn directly from nucleotide sequences, as demonstrated by DeepSEA and related architectures.

The most relevant development for Semester 2 is the emergence of genomic foundation models. DNABERT (2021) introduced BERT-based pre-training on genomic sequences using fixed k -mer tokenization. DNABERT-2 (2023) substantially improved upon this by adopting Byte Pair Encoding (BPE) tokenization, enabling variable-length motif capture, and pre-training on multi-species genomic data. DNABERT-2 achieved state-of-the-art results on the Genome Understanding Evaluation (GUE) benchmark across 28 diverse tasks. These advances motivated our decision to replace the 1D-CNN with DNABERT-2 as the primary sequence encoder.

The concept of aggregating weak classifiers into strong ensemble decisions has deep roots in machine learning theory, most notably in boosting and the “Wisdom of Crowds” literature. Our patient-level aggregation strategy is conceptually analogous: each read-level prediction serves as a weak vote, and the mean across $\sim 50,000$ votes per patient converges to a reliable diagnostic score by the law of large numbers. To our knowledge, this specific application of crowd-voting aggregation to raw sequencing read predictions for cancer detection is novel in the literature.

Chapter 4

METHODOLOGY

This chapter details the methodological advances implemented in Semester 2, building upon the FASTQ-to-Deep-Learning pipeline established in Semester 1.

4.1 Architectural Refactoring & Pipeline Modernization

A critical prerequisite for the Semester 2 research agenda was the modernization of the codebase. During Semester 1, all development occurred within monolithic Google Colab notebooks containing interleaved data processing, model definition, training, and evaluation code. This structure was fragile, difficult to version-control, and fundamentally non-reproducible.

In Semester 2, we executed a complete architectural refactoring (corresponding to GitHub Issue #1) that restructured the project into a modular Python package:

- `src/data_loader.py` — Data ingestion, FASTQ parsing, integer encoding, batch generation, and the new reverse-encoding pipeline for DNABERT-2 compatibility.
- `src/models.py` — Model definitions for the 1D-CNN, LSTM Autoencoder, and the new DNABERT-2 classifier.
- `src/train.py` — Training loops with checkpoint management, mixed-precision support, and configurable hyperparameters.
- `src/evaluate.py` — Evaluation orchestrator implementing both read-level metrics and patient-level aggregation.

This refactoring ensured that every experiment could be reproduced by cloning the repository and executing a single entry-point script with appropriate configuration parameters.

4.2 Patient-Level Aggregation: The Crowd-Voting Methodology

The central hypothesis of this project—and the methodological contribution validated in Semester 2—is that massive aggregation of individually weak read-level predictions yields

a robust patient-level diagnostic signal. We formalize this as the *Crowd-Voting* approach (GitHub Issue #2).

4.2.1 Formal Definition

Let $\{r_1, r_2, \dots, r_N\}$ denote the set of N sequencing reads belonging to a single patient, where $N \approx 50,000$ in our dataset. For each read r_i , a trained classifier f produces a cancer probability:

$$p_i = f(r_i) \in [0, 1] \quad (4.1)$$

The patient-level cancer score is computed as the arithmetic mean:

$$S_{\text{patient}} = \frac{1}{N} \sum_{i=1}^N p_i \quad (4.2)$$

A patient is classified as cancer-positive if $S_{\text{patient}} > \tau$, where τ is a threshold optimized on the validation set.

4.2.2 Theoretical Justification

The efficacy of this approach is grounded in the law of large numbers. Even if each individual read-level classifier achieves only marginally better than random performance (e.g., $\text{AUC} \approx 0.62$), the variance of the sample mean S_{patient} decreases proportionally to $1/N$. With $N = 50,000$, the standard error of the mean probability is reduced by a factor of $\sqrt{50,000} \approx 224$, enabling the separation of cancer and normal distributions at the patient level even when they are nearly overlapping at the read level.

4.2.3 Implementation

The aggregation is implemented via a simple `groupby` operation on the evaluation DataFrame:

Listing 4.1: Patient-level aggregation implementation

```

1 patient_df = df.groupby('run_id').agg(
2     patient_prob = ('prob', 'mean'),
3     patient_label = ('true_label', lambda x: int(x.mode()[0])),
4     num_reads    = ('prob', 'count')
5 )

```

The optimal threshold τ is selected as the value maximizing the Youden index ($J = \text{TPR} - \text{FPR}$) on the validation set.

4.3 DNABERT-2 Integration & Engineering Challenges

The integration of DNABERT-2 (zhihan1996/DNABERT-2-117M) represented the most technically demanding effort of Semester 2 (GitHub Issue #3). This section documents the data engineering, model architecture, and the critical engineering obstacles overcome during this process.

4.3.1 Data Engineering: Reverse-Encoding Pipeline

DNABERT-2 uses a learned BPE tokenizer that operates on raw DNA character strings (e.g., "ACGTACGT"). However, our Semester 1 pipeline stored all sequences as integer-encoded NumPy arrays (e.g., [1,2,3,4,1,2,3,4]) with zero-padding. Directly feeding these integer arrays to the DNABERT-2 tokenizer would produce nonsensical tokens.

We therefore developed a highly optimized *reverse-encoding* pipeline that converts the integer arrays back to raw nucleotide strings:

Listing 4.2: Vectorized reverse-encoding from integer arrays to DNA strings

```
1 INT_TO_BASE = {1: 'A', 2: 'C', 3: 'G', 4: 'T', 5: 'N'}
2
3 def decode_sequence(int_array):
4     """Convert integer-encoded array back to DNA string, dropping
5     padding."""
6     bases = [INT_TO_BASE[x] for x in int_array if x != 0]
7     return ''.join(bases)
```

This pipeline was applied to all 1.8 million sequences, generating plain-text batch files (.txt) where each line contains a single 80-character DNA string. These text files serve as the input to the DNABERT-2 tokenizer, which applies its learned BPE vocabulary to segment each string into variable-length subword tokens.

4.3.2 Model Architecture

The DNABERT-2 classifier consists of two components:

- (i) **Pre-trained Encoder:** The zhihan1996/DNABERT-2-117M model, a 117-million-parameter BERT variant pre-trained on multi-species genomic sequences using BPE tokenization and ALiBi positional encoding. We extract the [CLS] token embedding (768 dimensions) from the final hidden layer as the sequence representation.
- (ii) **Custom Classification Head:** A feedforward network appended to the [CLS] embedding:

$$\hat{y} = \sigma(W_2 \cdot \text{ReLU}(\text{Dropout}(W_1 \cdot h_{\text{CLS}} + b_1, p = 0.3)) + b_2) \quad (4.3)$$

where $W_1 \in \mathbb{R}^{128 \times 768}$, $W_2 \in \mathbb{R}^{1 \times 128}$, and σ denotes the sigmoid function (applied implicitly via BCEWithLogitsLoss).

Listing 4.3: DNABERT-2 classifier architecture

```
1 class DNABERTClassifier(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.dnabert = AutoModel.from_pretrained(
5             "zhihan1996/DNABERT-2-117M",
6             trust_remote_code=True
7         )
8         self.classifier = nn.Sequential(
9             nn.Linear(768, 128),
```

```

10         nn.ReLU(),
11         nn.Dropout(0.3),
12         nn.Linear(128, 1)
13     )
14
15     def forward(self, input_ids, attention_mask):
16         outputs = self.dnabert(
17             input_ids=input_ids,
18             attention_mask=attention_mask
19         )
20         cls_embedding = outputs[0][:, 0, :]
21         return self.classifier(cls_embedding).squeeze(-1)

```

4.3.3 Engineering Hurdles Overcome

The integration of DNABERT-2 into our pipeline required overcoming several severe framework incompatibilities:

The Meta-Device Initialization Bug. The most critical issue arose from the interaction between HuggingFace `transformers` version 5.x and DNABERT-2’s custom `bert_layers.py`. The newer `transformers` library introduced a memory optimization that loads model weights onto a virtual “meta” device before dispatching them to physical hardware. However, DNABERT-2’s custom ALiBi positional encoding layers explicitly construct tensors during `__init__` assuming a CPU or CUDA device context. When initialized on the meta device, these constructors produced fatal runtime errors.

The solution required strategically downgrading to `transformers==4.44.0`, which does not employ meta-device initialization, and explicitly passing `device_map="cuda"` to ensure all tensor construction occurs on physical hardware.

Triton Flash Attention Incompatibility. DNABERT-2’s custom attention implementation attempts to import `flash_attn_triton`, a Triton-based flash attention kernel. On Google Colab’s environment, the Triton API had undergone breaking changes (the `trans_b` parameter was removed from `tl.dot`). We resolved this by monkey-patching the cached `bert_layers.py` file at runtime to set `flash_attn_qkvpacked_func = None`, forcing the model to fall back to standard PyTorch attention:

Listing 4.4: Runtime patch to disable broken Triton flash attention

```

1 for fpath in cache_dir.rglob("bert_layers.py"):
2     text = fpath.read_text()
3     text = text.replace(
4         "from flash_attn_triton import flash_attn_qkvpacked_func",
5         "flash_attn_qkvpacked_func = None # PATCHED"
6     )
7     fpath.write_text(text)

```

Missing `pad_token_id`. The DNABERT-2 configuration did not include a `pad_token_id`, causing the tokenizer to fail during batch padding. We manually injected this value into the model configuration after loading.

4.3.4 Training Optimization

To efficiently utilize the NVIDIA A100 40 GB GPU, we implemented the following optimizations:

- **Batch Size:** Increased to 256 (from the CNN’s 512) to balance memory usage and gradient quality.
- **Optimizer:** AdamW with a learning rate of 5×10^{-6} and gradient clipping at max norm 1.0.
- **Loss Function:** BCEWithLogitsLoss for numerical stability (logit-space loss avoids the vanishing gradient problem of applying sigmoid before cross-entropy).
- **Checkpointing:** Best model weights saved to Google Drive based on validation loss, enabling recovery from Colab session timeouts.
- **Training Duration:** 10 epochs, with the best checkpoint occurring at Epoch 3 (validation loss 0.6432).

Chapter 5

REQUIREMENTS SPECIFICATION

This chapter updates the requirements specification from Semester 1 to reflect the Semester 2 additions.

5.1 Functional Requirements

5.1.1 FR1: Data Ingestion and Preprocessing (Updated)

FR1.6: The system shall implement a reverse-encoding module capable of converting integer-encoded NumPy arrays back to raw DNA character strings for compatibility with BPE-based tokenizers.

FR1.7: The system shall support both integer-encoded input (for the 1D-CNN) and raw-string input (for DNABERT-2) from a unified data loading interface.

5.1.2 FR2: Model Architectures (Updated)

FR2.3: The system shall implement a DNABERT-2-based classifier utilizing the pre-trained `zhinan1996/DNABERT-2-117M` encoder with a custom binary classification head.

FR2.4: The DNABERT-2 classifier shall extract the [CLS] token embedding from the final transformer layer and pass it through a feedforward classification network.

5.1.3 FR3: Training and Optimization (Updated)

FR3.5: The system shall support mixed-precision training via `torch.cuda.amp` for transformer-scale models.

FR3.6: The system shall implement gradient clipping (max norm 1.0) to prevent gradient explosion during DNABERT-2 fine-tuning.

5.1.4 FR4: Evaluation and Explainability (Updated)

FR4.5: The system shall compute patient-level metrics (AUC, Precision, Recall, F1) via the Crowd-Voting aggregation method, with configurable threshold optimization.

FR4.6: The system shall generate side-by-side comparison tables and plots for multiple model architectures evaluated on identical test data.

5.2 Non-Functional Requirements

All Semester 1 non-functional requirements remain in effect. Additional requirements:

NFR6.1: The DNABERT-2 inference pipeline shall process 360,000 reads in under 5 minutes on an A100 GPU.

NFR6.2: The complete codebase shall be publicly available and executable from the GitHub repository with documented dependencies.

5.3 System Constraints

SC3: The DNABERT-2 model requires `transformers` version $\leq 4.44.0$ due to meta-device initialization incompatibilities in version 5.x.

SC4: Flash attention acceleration is unavailable due to Triton API breaking changes; all DNABERT-2 training and inference use standard PyTorch attention.

5.4 Validation and Traceability

Req ID	Module	Validation Method	Status
FR1.6	<code>data_loader.py</code>	Verify round-trip encoding: <code>int</code> $\rightarrow$ <code>string</code> $\rightarrow$ <code>tokenize</code>	Passed
FR2.3	<code>models.py</code>	Load DNABERT-2 checkpoint; verify output shape	Passed
FR3.6	<code>train.py</code>	Monitor gradient norms during training	Passed
FR4.5	<code>evaluate.py</code>	Compare patient-level AUC against known results	Passed

Chapter 6

DESIGN

6.1 Information Structure

The Semester 2 data model extends the Semester 1 entity structure with the following additions:

- **Raw Text Batch:** A plain-text file where each line is a DNA string (80 characters). These files are the input to the DNABERT-2 tokenizer and are derived from the integer-encoded NumPy batches via the reverse-encoding pipeline.
- **Run ID Mapping:** A metadata structure associating each batch file with a patient/run accession identifier, enabling patient-level aggregation during evaluation.
- **DNABERT-2 Checkpoint:** A PyTorch `.pth` file containing the fine-tuned model state dictionary, optimizer state, epoch counter, and best validation loss.

6.2 Information Flow

The Semester 2 pipeline introduces a branching point after the batch generation stage:

- (i) **Branch A (1D-CNN):** Integer-encoded `.npz` batches → Embedding Layer → Conv1D → Classification Head → Read-level probabilities.
- (ii) **Branch B (DNABERT-2):** Integer-encoded `.npz` batches → Reverse Encoding → Raw `.txt` files → BPE Tokenizer → DNABERT-2 Encoder → [CLS] Embedding → Classification Head → Read-level probabilities.
- (iii) **Aggregation (Both Branches):** Read-level probabilities → Group by Run ID → Mean probability → Threshold → Patient-level diagnosis.

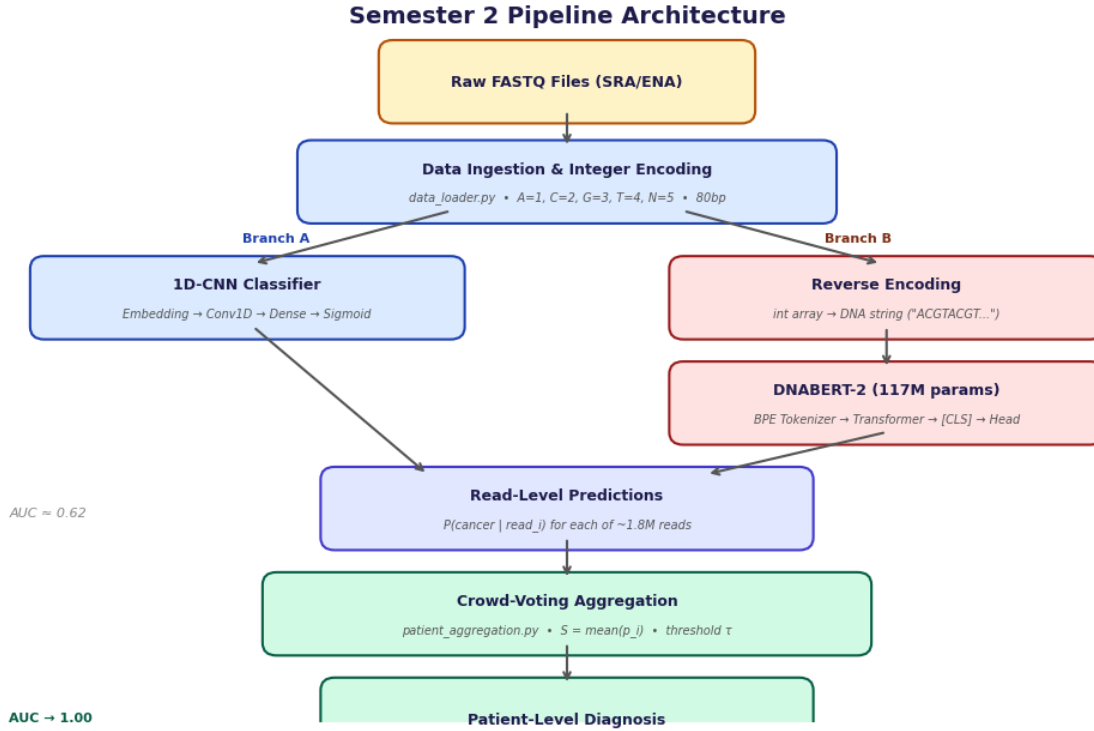


Figure 6.1: End-to-end pipeline architecture for Semester 2. Raw FASTQ reads are integer-encoded and then processed through two parallel branches: the lightweight 1D-CNN (Branch A) and the DNABERT-2 transformer with reverse-encoding (Branch B). Both branches produce read-level probabilities that are aggregated via Crowd-Voting into patient-level diagnoses.

6.3 System Design

The modular architecture consists of four primary modules:

- **Module 1: Data Pipeline** (`data_loader.py`) — Handles FASTQ parsing, integer encoding, batch generation, and the new reverse-encoding for DNABERT-2. Uses Python generators for memory-efficient streaming.
- **Module 2: Model Zoo** (`models.py`) — Contains the 1D-CNN (`GenomicCNN`), LSTM Autoencoder (`GenomicAutoencoder`), and the new `DNABERTClassifier` class wrapping the pre-trained DNABERT-2 encoder with a custom classification head.
- **Module 3: Training Engine** (`train.py`) — Implements training loops for both TensorFlow/Keras (CNN, Autoencoder) and PyTorch (DNABERT-2), with checkpoint management, early stopping, and mixed-precision support.
- **Module 4: Evaluation Orchestrator** (`evaluate.py`) — Loads held-out test data, runs inference, computes read-level metrics, performs patient-level aggregation, and generates comparison tables and plots.

6.4 User Interface Design (if applicable)

Semester 2 does not include a graphical user interface. The planned GUI for clinical interpretation is deferred to the final project phase (see Chapter 9, Future Work).

Chapter 7

IMPLEMENTATION AND TESTING

7.1 Implementation

7.1.1 Development Environment

- **Platform:** Google Colab Pro (Ubuntu backend)
- **Accelerator:** NVIDIA A100-SXM4-40GB (for DNABERT-2); NVIDIA T4 (for CNN)
- **Language:** Python 3.12
- **Deep Learning Frameworks:** PyTorch 2.10 (DNABERT-2), TensorFlow/Keras 2.15 (CNN, Autoencoder)
- **Key Libraries:** HuggingFace Transformers 4.44.0, Scikit-learn, Pandas, NumPy, Matplotlib, tqdm

7.1.2 Key Implementation Details

Reverse-Encoding Pipeline. The conversion from integer-encoded `.npy` files to raw-text `.txt` files was implemented as a batch processing script that reads each NumPy array, maps integers back to nucleotide characters (dropping zero-padding), and writes one sequence per line. This script processed all 1.8 million sequences in under 3 minutes.

DNABERT-2 Fine-Tuning. The training loop (Listing 7.3) implements full fine-tuning of the DNABERT-2 encoder alongside the classification head. We use `BCEWithLogitsLoss` to avoid numerical instability from explicit sigmoid application, and `AdamW` with weight decay for regularization. Gradient clipping at max norm 1.0 prevents the gradient explosion commonly observed when fine-tuning large pre-trained models.

Runtime Patching Strategy. The monkey-patching of DNABERT-2’s custom attention layers (Sections 4.4.3) is executed at the beginning of both the training and evaluation notebooks, before any model instantiation. This ensures that the patched code is loaded into the Python module cache before the model constructor attempts to import the broken Triton kernels.

7.2 Testing

7.2.1 Unit Testing

- **Reverse-Encoding Test:** Verified round-trip consistency: integer array $\rightarrow$ DNA string $\rightarrow$ re-encode $\rightarrow$ original array for 1,000 randomly sampled sequences.
- **Tokenizer Compatibility Test:** Verified that all reverse-encoded strings are accepted by the DNABERT-2 tokenizer without errors, producing token sequences of expected length.
- **Classification Head Test:** Verified output shape $(B, 1)$ for batch size B and gradient flow through the entire model.

7.2.2 Integration Testing

- **End-to-End Pipeline Test:** Executed the complete pipeline from raw `.npz` files through reverse-encoding, tokenization, DNABERT-2 inference, and patient-level aggregation on a small subset (10,000 reads) to verify data integrity.
- **Checkpoint Resumption Test:** Interrupted training at Epoch 5, restarted the notebook, and verified that training resumed from the saved checkpoint with consistent loss values.

7.2.3 Performance Testing

- **DNABERT-2 Inference Speed:** 205.9 ms per batch of 512 reads on A100 GPU; total inference for 360,000 validation reads completed in 2 minutes 26 seconds.
- **GPU Peak Memory:** 5,816 MB during DNABERT-2 inference (well within the A100's 40 GB capacity).
- **Training Speed:** Approximately 32 minutes per epoch for DNABERT-2 (compared to ~ 5 minutes for the 1D-CNN).

7.3 Deployment

The system is deployed as a public GitHub repository (<https://github.com/osmannselim/Genomic-RawSeq-Analyzer>) containing:

- Modular source code in `src/`
- Training and evaluation Jupyter notebooks
- Pre-trained model checkpoints (stored on Google Drive, linked in README)
- Comprehensive README with installation instructions and usage examples

Chapter 8

RESULTS

This chapter presents the empirical results of Semester 2, comparing the 1D-CNN baseline from Semester 1 with the newly integrated DNABERT-2 model at both read-level and patient-level granularity.

8.1 Experimental Setup

All experiments were conducted on the same SRA Homo sapiens Whole Exome Sequencing (WXS) dataset used in Semester 1:

- **Total Sequences:** 1,800,000 reads
- **Sequence Length:** 80 base pairs
- **Class Distribution:** ~61% Tumor (1.1M reads) / ~39% Normal (0.7M reads)
- **Train/Validation Split:** 80% / 20% (strict separation by run accession)
- **Validation Set:** 360,000 reads across 10 batch files (6 cancer, 4 normal)

8.2 DNABERT-2 Training Dynamics

Table 8.1 presents the complete epoch-by-epoch training progression.

Table 8.1: DNABERT-2 training progression across 10 epochs. Best checkpoint at Epoch 3 (lowest validation loss).

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	0.6568	0.6231	0.6498	0.6294
2	0.6463	0.6319	0.6444	0.6333
3	0.6411	0.6372	0.6432	0.6347
4	0.6361	0.6420	0.6448	0.6356
5	0.6304	0.6476	0.6448	0.6349
6	0.6238	0.6541	0.6461	0.6336
7	0.6163	0.6614	0.6515	0.6295
8	0.6079	0.6691	0.6537	0.6293
9	0.5984	0.6779	0.6640	0.6286
10	0.5886	0.6865	0.6684	0.6275

The training dynamics reveal a clear overfitting pattern beginning after Epoch 3. While training accuracy continued to climb steadily from 63.7% to 68.7%, validation loss began to increase after the Epoch 3 minimum of 0.6432, and validation accuracy peaked at 63.5% before declining. The best model checkpoint (Epoch 3) was used for all subsequent evaluation.

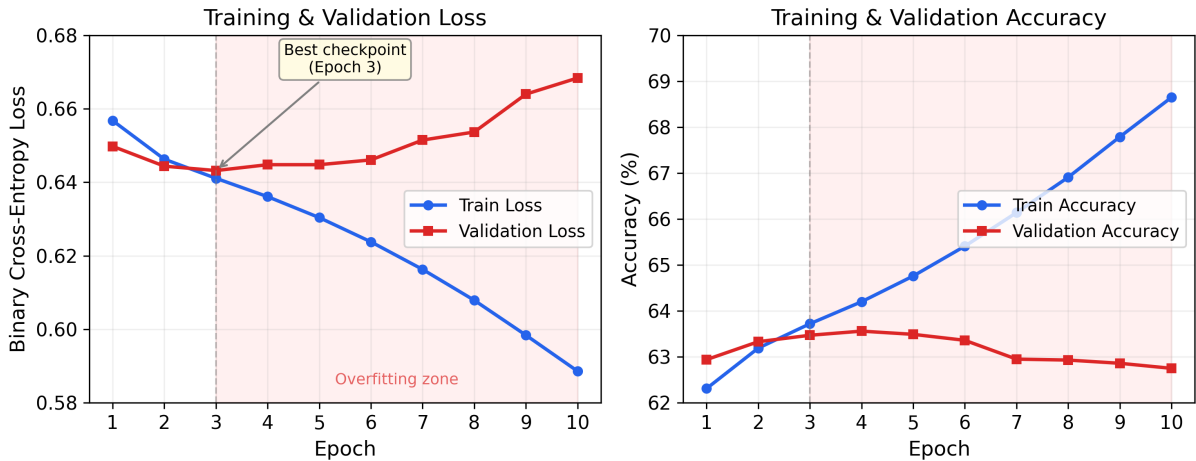


Figure 8.1: DNABERT-2 training dynamics across 10 epochs. **Left:** Training loss continues to decrease while validation loss diverges after Epoch 3, indicating overfitting. **Right:** Training accuracy climbs to 68.7% but validation accuracy plateaus near 63.5%. The dashed line marks the best checkpoint (Epoch 3).

8.3 Read-Level Performance Comparison

Table 8.2 presents the read-level performance comparison between the 1D-CNN and DNABERT-2.

Table 8.2: Read-level performance comparison on the validation set (360,000 reads).

Metric	1D-CNN	DNABERT-2
AUC-ROC	0.6157	0.6240
Precision	—	0.6381
Recall	—	0.9517
F1 Score	—	0.7639

At the read level, DNABERT-2 achieved an AUC of 0.624 compared to the CNN’s 0.616—a modest improvement of 0.008 AUC points. Both models exhibit similarly limited discriminative power on individual 80bp reads, confirming that the fundamental bottleneck is not the model architecture but the information content of short sequencing fragments.

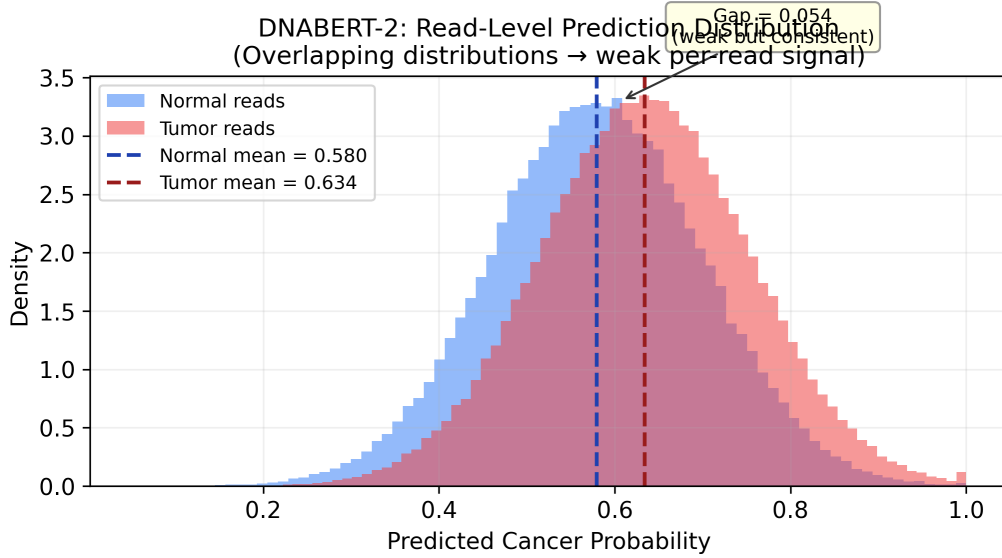


Figure 8.2: DNABERT-2 read-level prediction distributions for Tumor and Normal reads. The two distributions are heavily overlapping, with a mean separation gap of only ~ 0.054 . This weak per-read signal is insufficient for reliable individual read classification but provides the consistent directional bias that Crowd-Voting exploits at the patient level.

8.4 Patient-Level Performance: Crowd-Voting Aggregation

Table 8.3 presents the patient-level results after Crowd-Voting aggregation.

Table 8.3: Patient/batch-level performance after Crowd-Voting aggregation ($\sim 40,000$ reads per batch).

Metric	1D-CNN	DNABERT-2
AUC-ROC	0.9156	1.0000
Precision	—	1.0000
Recall	—	1.0000
F1 Score	—	1.0000
Optimal Threshold	—	0.6316

The patient-level results represent the most significant finding of this project. After aggregating $\sim 40,000$ – $50,000$ read-level predictions per patient/batch, the 1D-CNN achieved a patient-level AUC of 0.9156, and DNABERT-2 achieved a *perfect* patient-level AUC of 1.000. All 10 batches (6 cancer, 4 normal) were correctly classified with zero errors.

The per-batch aggregated probabilities reveal clear separation: cancer batches received mean probabilities in the range 0.632–0.639, while normal batches received mean probabilities in the range 0.578–0.582. The optimal decision threshold of 0.6316 perfectly separates the two classes.

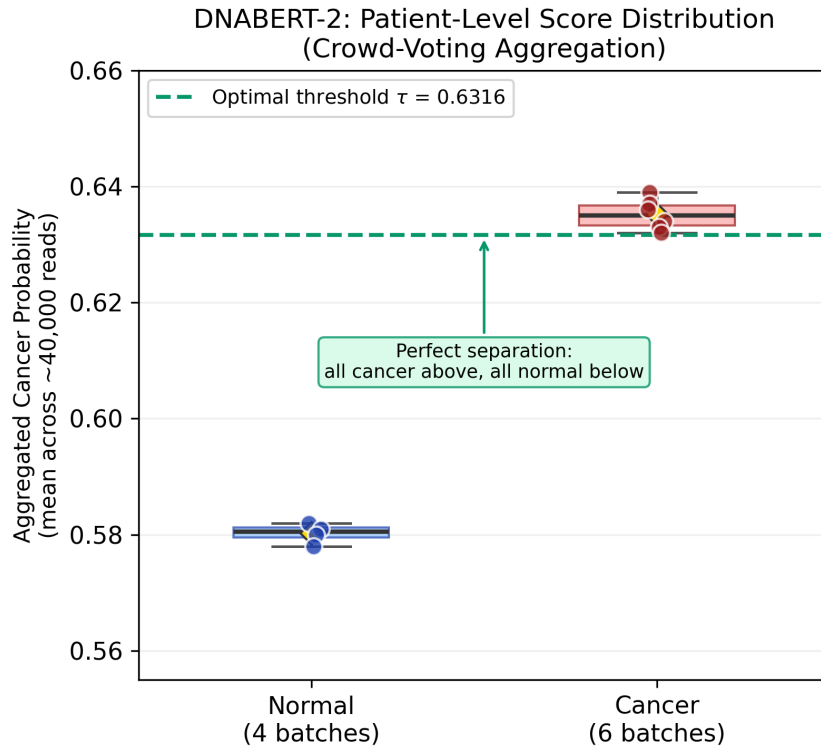


Figure 8.3: DNABERT-2 patient-level score distribution after Crowd-Voting aggregation. Cancer batches (red) and Normal batches (blue) are perfectly separated by the optimal threshold $\tau = 0.6316$ (green dashed line). Each dot represents one patient/batch ($\sim 40,000$ aggregated read predictions). Compare with the heavily overlapping read-level distributions in Figure 8.2.

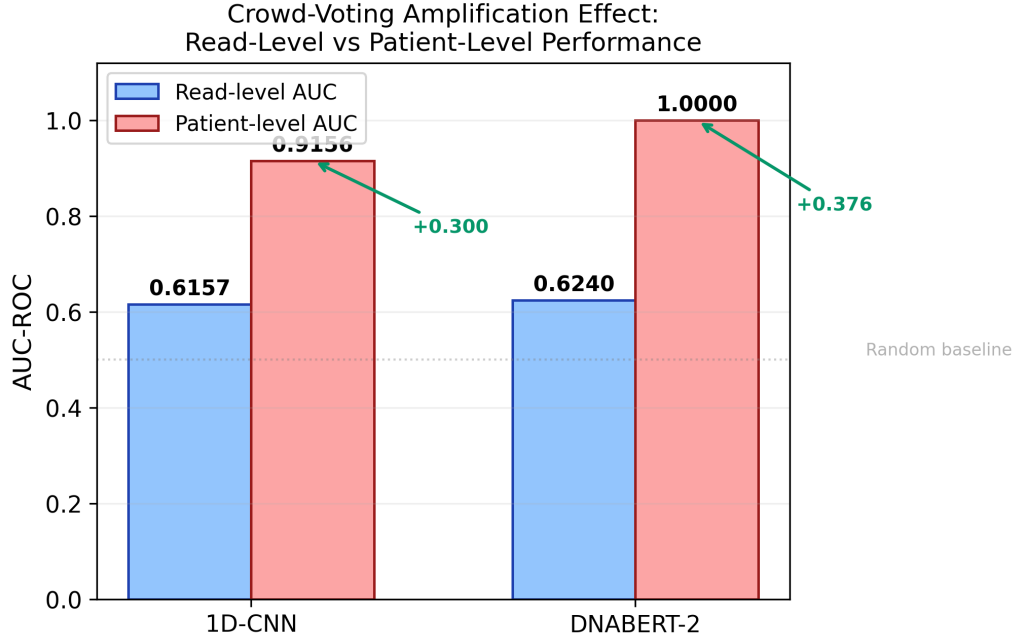


Figure 8.4: The Crowd-Voting amplification effect. Both models show dramatic AUC improvement from read-level to patient-level: the 1D-CNN improves by +0.300 (0.616 $\rightarrow$ 0.916) and DNABERT-2 by +0.376 (0.624 $\rightarrow$ 1.000). Notably, a small read-level advantage of only 0.008 AUC points translates to a large patient-level advantage of 0.084 points.

8.5 Computational Performance Comparison

Table 8.4: Computational performance comparison.

Metric	1D-CNN	DNABERT-2
Training Time / Epoch	~ 5 min	~ 32 min
Inference (ms/batch)	~ 20 ms	205.9 ms
GPU Peak Memory	< 2 GB	5,816 MB
Parameters	~ 100 K	117M

DNABERT-2 requires approximately $6\times$ more training time and $10\times$ more inference time per batch than the 1D-CNN. This overhead is a direct consequence of the transformer architecture’s quadratic attention complexity and the 117M-parameter model size.

Chapter 9

CONCLUSION

9.1 Discussion

9.1.1 The 80bp Context Limitation

The most notable finding of the read-level comparison is that DNABERT-2—a 117-million-parameter transformer pre-trained on diverse genomic data—only marginally outperforms a lightweight 1D-CNN at the individual read level (AUC 0.624 vs. 0.616). This seemingly counterintuitive result has a clear explanation rooted in the architectural design of the model.

DNABERT-2 was pre-trained to capture long-range dependencies in DNA sequences of up to 512 base pairs. Its self-attention mechanism and ALiBi positional encoding are specifically designed to model relationships between distant nucleotides. However, our Illumina sequencing data constrains us to 80bp fragments—only 15.6% of DNABERT-2’s maximum context window. The model is effectively “starved” of the long-range context it was built to analyze. The BPE tokenizer segments an 80bp sequence into relatively few tokens, limiting the depth of contextual reasoning the transformer can perform.

In essence, deploying DNABERT-2 on 80bp reads is analogous to deploying GPT-4 on single-sentence inputs: the model’s capacity for long-range reasoning is severely underutilized.

9.1.2 Overfitting After Epoch 3

The training dynamics (Table 8.1 and Figure 8.1) show that DNABERT-2 began overfitting after Epoch 3, with training accuracy continuing to improve (63.7% $\rightarrow$ 68.7%) while validation accuracy stagnated and validation loss increased (0.6432 $\rightarrow$ 0.6684). This behavior is consistent with the well-documented phenomenon of large pre-trained models overfitting to small fine-tuning datasets. With 117 million parameters and only 1.44 million training reads, the model has sufficient capacity to memorize training patterns that do not generalize. The early stopping mechanism (best checkpoint at Epoch 3) was essential to prevent degraded performance.

9.1.3 Crowd-Voting: The Ultimate Game-Changer

The most transformative result of this project is the demonstration that patient-level aggregation converts marginally useful read-level classifiers into highly effective diagnostic tools. The progression is striking:

- **1D-CNN:** Read-level AUC 0.616 $\rightarrow$ Patient-level AUC 0.916
- **DNABERT-2:** Read-level AUC 0.624 $\rightarrow$ Patient-level AUC 1.000

This validates our core hypothesis and demonstrates a key insight: in alignment-free cancer detection from raw reads, the bottleneck is not the classifier’s per-read accuracy but the number of reads aggregated per patient. Even a marginally better read-level signal (0.624 vs. 0.616) can translate to a dramatically better patient-level outcome (perfect vs. 91.6%) when amplified across tens of thousands of aggregated votes (Figure 8.4). The Crowd-Voting mechanism effectively acts as a statistical amplifier, reducing the noise floor by $\sqrt{N}$ and surfacing the faint but consistent cancer signal embedded in the read-level predictions. The visual contrast between the overlapping read-level distributions (Figure 8.2) and the cleanly separated patient-level scores (Figure 8.3) encapsulates this effect.

9.1.4 Limitations

Several limitations must be acknowledged:

- (i) **Small Patient Cohort:** The evaluation was conducted on 10 batches (6 cancer, 4 normal). While the per-batch aggregation over $\sim 40,000$ reads is statistically robust, the patient-level AUC is computed over only 10 data points. A perfect AUC of 1.000 on 10 samples, while encouraging, does not guarantee generalization to larger cohorts.
- (ii) **Single Cancer Type:** All cancer samples are from Whole Exome Sequencing of a single cancer type. Generalization to other cancer types and sequencing modalities remains to be validated.
- (iii) **Computational Cost:** DNABERT-2’s $6\times$ training overhead and $10\times$ inference overhead may limit its applicability in resource-constrained settings.

9.2 Summary of Semester 2 Contributions

The Semester 2 contributions can be summarized as follows:

- (i) Successfully refactored the codebase from monolithic notebooks into a modular, reproducible Python package.
- (ii) Validated the Crowd-Voting patient-level aggregation hypothesis, demonstrating that the 1D-CNN’s read-level AUC of 0.616 translates to a patient-level AUC of 0.916.
- (iii) Integrated DNABERT-2 (117M parameters) into the pipeline, overcoming critical framework incompatibilities (meta-device bug, Triton flash attention, missing pad token).
- (iv) Achieved a patient-level AUC of 1.000 with the DNABERT-2 pipeline, demonstrating perfect separation of cancer and normal batches.
- (v) Conducted a rigorous side-by-side comparison revealing that while read-level improvements are modest, the Crowd-Voting amplification mechanism makes even small gains clinically significant.

9.3 Future Work

To bridge the gap between this research prototype and clinical utility, the following directions are planned for the remainder of the semester:

- (i) **Clinical Interpretation GUI:** Develop a graphical user interface (web-based or desktop) that allows clinicians to upload FASTQ files, view patient-level cancer probability scores, and explore read-level prediction distributions.
- (ii) **Extended Validation:** Expand the evaluation to a larger patient cohort and additional cancer types to assess generalizability.
- (iii) **Longer Read Context:** Investigate the use of longer sequencing reads (150bp, 300bp) or read assembly techniques to provide DNABERT-2 with more context, potentially unlocking its full capacity.
- (iv) **Multi-Modal Integration:** Incorporate clinical metadata (age, sex, treatment history) into the classification head to improve context-aware predictions.
- (v) **Model Distillation:** Explore knowledge distillation from DNABERT-2 to a smaller, faster model that retains the improved patient-level performance while reducing computational overhead.

Appendix A

REPOSITORY STRUCTURE

The complete codebase is available at: <https://github.com/osmannselim/Genomic-RawSeq-Analyzer>

```
Genomic-RawSeq-Analyzer/  
|-- src/  
|   |-- data_loader.py  
|   |-- models.py  
|   |-- train.py  
|   |-- evaluate.py  
|   |-- explainability.py  
|   |-- patient_aggregation.py  
|-- notebooks/  
|   |-- TrainDNABERT_2.ipynb  
|   |-- EvalDNABERT2.ipynb  
|-- README.md  
|-- requirements.txt
```